The separation of Prometheus and the Alertmanager, as shown in figure 7.2, allows for flexibility and scalability. For example, you can scale or restart either of the tools without interrupting the other.

> **TIP** From an operations perspective, at the end of the day, you want to avoid alert fatigue (https://en.wikipedia.org/wiki/Alarm_fatigue). That is, you want to minimize the notifications to act on while making sure you don't miss something substantial. Imagine the case in which you're responsible for a Kubernetes cluster with multiple namespaces. If there are data plane problems with the nodes not being reachable or rebooting, you only want to get one notification, rather than, say, one notification for every single pod in the cluster.

Now, let's dive deeper into how alerting rules and the Alertmanager work.

PROMETHEUS ALERTING RULES

Starting with Prometheus, we now work our way through defining and handling alerts. In the Prometheus configuration, you define the alerts using alerting rules (http://mng.bz/XNj9). An alerting rule defines the condition you want Prometheus to evaluate. A concrete example looks as follows (adapted from the excellent reference site Awesome Prometheus Alerts (https://awesome-prometheus-alerts.grep.to/), which provides a large collection of curated alert examples you can use):

The name of the alert. The convention is CamelCase.

The condition to evaluate—a PromQL expression Prometheus executes

Optionally, you can specify for how long the condition needs to be met before sending the alert to the Alertmanager.

Key–value metadata for filtering and routing in the Alertmanager

Key–value context for direct human consumption (content of the notification) using templating

```
- alert: PrometheusAllTargetsMissing
  expr: sum by (job) (up) == 0
  for: 0m
  labels:
    severity: critical
  annotations:
    summary: Prometheus all targets missing {{ $labels.instance }}
    description: "A Prometheus job has no alive targets.\n
                  VALUE = {{ $value }}\n  LABELS = {{ $labels }}"
```

While the `for` field is optional, you should consider setting it. The reason is that, in cloud-native systems, you're dealing with a lot of ephemeral components, network connectivity issues, and timeouts or retries. You don't want a trigger-happy alert spamming you while, really, it was just a temporary issue with the telemetry, for example. Let's say we would have chosen `for: 3m` in the preceding example. In this case, Prometheus initially considers the alert in the pending state when the condition is met for the first time. The alert is considered to be in the firing state and sent to the Alertmanager if the condition persists for 3 minutes. In the preceding snippet, we used `for: 0m`, and because of this, the alert went directly to firing.